

Relatório 2º projecto ASA 2025/2026

Grupo: AL054

Aluno(s): Duarte Graça (113395) e Carlota Vicente (115090)

Descrição da Solução

Explicação da solução:

- Ao ler as K ligações de cruzamentos no input, registamos num vetor adjacente de *unsigned ints* todas as ligações que cada cruzamento tem. Deste modo ao percorrer todo o vetor de adjacências conseguimos construir o grafo.
- Aplica-se o algoritmo de Khan ao vetor de adjacentes, de modo a obter a ordem topológica dos cruzamentos.
- De seguida, iteramos sobre cada cruzamento em ordem lexicográfica, começando em $i = 1$, e num *nested for* iteramos sobre cada cruzamento em ordem topológica, começando em $j = 1$. Para tal, criamos o vetor *caminhosOrigemI* que armazena o número de caminhos existentes a partir da origem, i , e o vetor *isValid* que armazena qual era o i quando cada número de caminhos foi armazenado em *caminhosOrigemI*.

```
criar rotCam[0..(m2-m1+1)]
criar caminhosOrigemI[1..N]
criar isValid[1..N]
stamp ← 0
para i = 1 até N
    stamp ← stamp + 1
    isValid[i] ← stamp
    caminhosOrigemI[i] ← 1
```
- A cada iteração de j , adicionamos o valor de índice j da ordem topológica do vetor *caminhosOrigemI* (quando válido), a todos os adjacentes do valor de índice j da ordem topológica. Para prevenir o *overflow* e acelerar o programa, se com esta adição o valor desta posição no vetor for maior que o número de camiões, subtraímos este pelo número de camiões (essencialmente, fazemos o módulo da divisão entre o valor e o número de camiões).

```
para j = 1 até N
    u ← top[j]
    val ← GET-VALUE(u)
    se val = 0 então
        continuar
    para cada adjacente em adj[u]
        ADD-VALUE(adjacente, val)
```
- Seguidamente, no *loop* exterior, criamos outro *nested for* começando em $k=1$ que percorre todo o vetor *caminhosOrigemI* e atribui a cada par de cruzamentos i e k um camião *numCam* baseado no número de caminhos possíveis entre ambos, armazenando cada par no vetor *rotCam* no índice *numCam-m1*.

```
para k = 1 até N
    se k = i ou isValid[k] ≠ stamp então
        continuar
    numCam ← caminhosOrigemI[k]
    se numCam = M então
        numCam ← 1
    senão
        numCam ← numCam + 1
    se m1 < numCam < m2 então
        adicionar par (i, k) ao fim de rotCam[numCam - m1]
```

Relatório 2º projecto ASA 2025/2026

Grupo: AL054

Aluno(s): Duarte Graça (113395) e Carlota Vicente (115090)

- Depois de todas iterações, percorremos *rotCam* e imprimimos os caminhos pertencentes a cada camião.

```
para i = 0 até (m2-m1)
    cam ← m1 + i
    imprimir "C" cam (sem newline final)
    para cada par (A,B) em rotCam[i], pela ordem guardada
        imprimir " " A " ," B
    imprimir newline
```

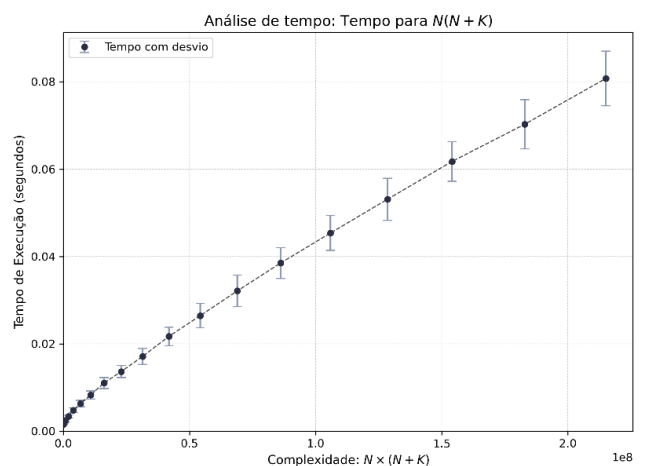
Análise Teórica da Solução Proposta

- Leitura dos dados de entrada e construção do grafo: $O(K)$.
A leitura das primeiras linhas de input é imediata, sendo o número de ligações, K , o único que varia.
- Calcular ordem topológica: $O(N + K)$.
Colocar o *indegree* tendo em conta os adjacentes é $O(N + K)$, colocar na queue todos os elementos com *indegree* 0 é $O(N)$, retirar elemento a elemento da queue e voltar a colocar na *queue* os com *indegree* = 0 é $O(N + K)$, logo a complexidade final é $O(N + K)$.
- Algoritmo de cálculo dos caminhos e camião para a rota: $O(N * (N+K))$.
Loop exterior é $O(N)$, enquanto os loops interiores são respetivamente $O(N + K)$ e $O(N)$ o que se pode reduzir a $O(N + K)$, logo a complexidade final é de $O(N * (N + K))$, caso o grafo seja muito denso K tende para N^2 , fazendo a complexidade poder ser até $O(N^3)$.
- Complexidade global: $O(N * (N+K))$, pois o maior grau é a que prevalece.

Avaliação Experimental da Solução Proposta

Testámos o nosso código com 19 inputs de 1000 camiões com densidade de ligações a 50%, N varia entre 50 e 950, sendo executados 10 testes por N . Utilizamos uma *seed* aleatória.

O eixo dos XX representa a complexidade teórica ($N*(N + K)$) e o eixo dos YY o tempo de execução em segundos. É possível observar que o tempo de execução cresce linearmente com o crescimento teórico esperado, confirmando que, de facto, a solução tem complexidade $\approx O(N*(N + K))$.



Tamanho n	50	100	150	200	250	300	350	400	450	500	550	600	650	700	750	800	850	900	950
Tempo médio (ms)	1,5	1,9	2,5	3,4	4,8	6,3	8,3	11	13,6	17,1	21,7	26,4	32,1	38,5	45,4	53,1	62	70,3	80,8

Referências

GeeksforGeeks. "Topological Sorting", 12 May 2013, [Hiperligação](#) Accessed 19 Dec. 2025.